Course Discussions

Solution: Strings Interleaving

Problem Statement

Given three strings 'm', 'n', and 'p', write a method to find out if 'p' has been formed by interleaving 'm' and 'n'. 'p' would be considered interleaving 'm' and 'n' if it contains all the letters from 'm' and 'n' and the order of letters is preserved too.

Example 1:

```
Input: m="abd", n="cef", p="abcdef"
Output: true
Explanation: 'p' contains all the letters from 'm' and 'n' and preserves
their order too.
```

Example 2:

```
Input: m="abd", n="cef", p="adcbef"
Output: false
Explanation: 'p' contains all the letters from 'm' and 'n' but does not
preserve the order.
```

Example 3:

```
Input: m="abc", n="def", p="abdccf"
Output: false
Explanation: 'p' does not contain all the letters from 'm' and 'n'.
```

Example 4:

```
Input: m="abcdef", n="mnop", p="mnaobcdepf"
Output: true
Explanation: 'p' contains all the letters from 'm' and 'n' and preserves
their order too.
```

Basic Solution

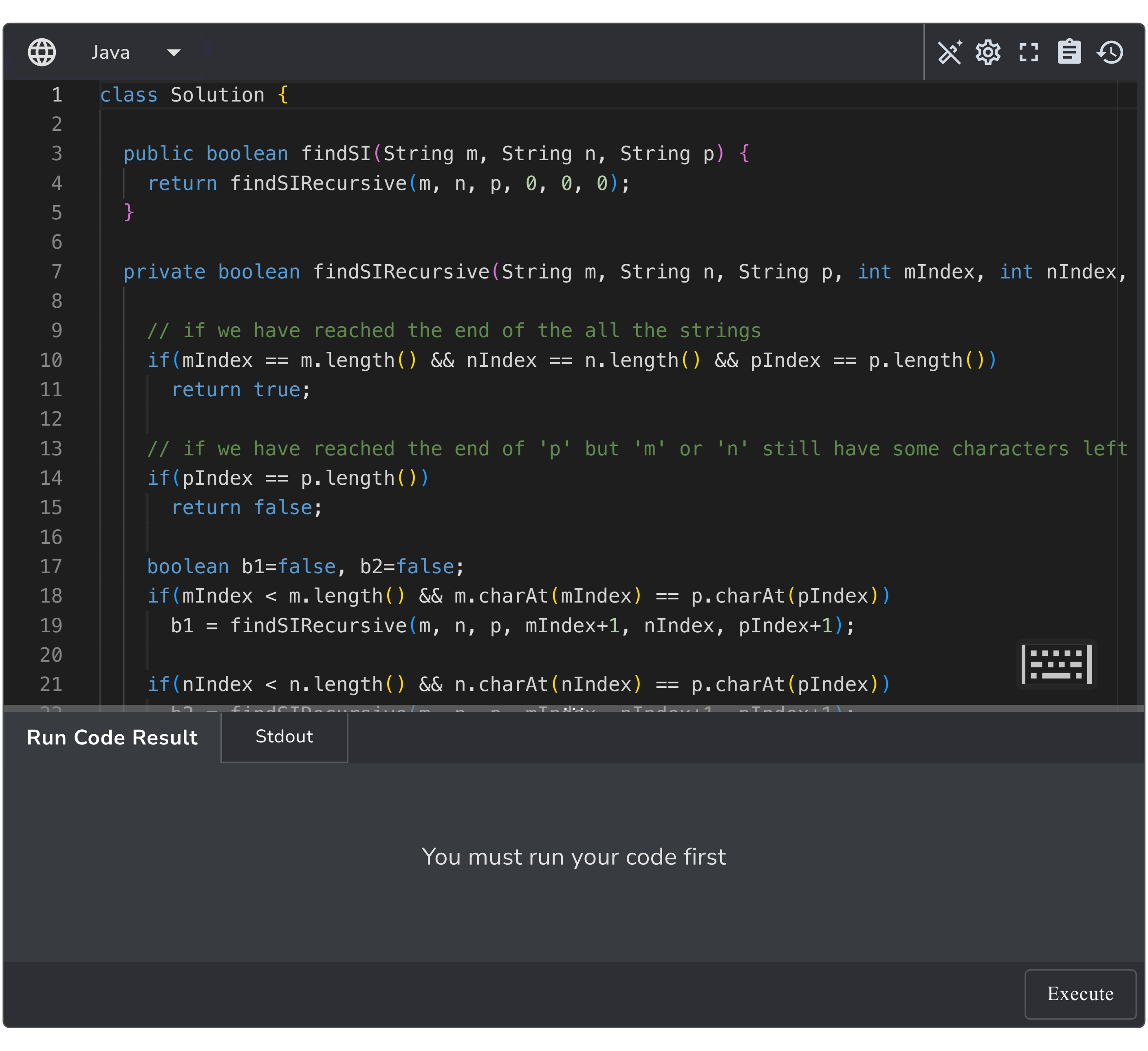
The problem follows the `"Longest Common Subsequence" (LCS)` pattern and has some similarities with `"Subsequence Pattern Matching"`.

A basic brute-force solution could be to try matching 'm' and 'n' with 'p' one letter at a time. Let's assume `mIndex`, `nIndex`, and `pIndex` represent the current indexes of 'm', 'n', and 'p' strings respectively. Therefore, we have two options at any step:

1. If the letter at `mIndex` matches with the letter at `pIndex`, we can recursively match for the remaining lengths of 'm' and 'p'.
2. If the letter at `nIndex` matches with the letter at 'pIndex', we can recursively match for the remaining lengths of 'n' and 'p'.

Code

Here is the code:



```java
class Solution {

  public boolean findSI(String m, String n, String p) {
    return findSIRecursive(m, n, p, 0, 0, 0);
  }

  private boolean findSIRecursive(String m, String n, String p, int mIndex, int nIndex,

    // if we have reached the end of the all the strings
    if(mIndex == m.length() && nIndex == n.length() && pIndex == p.length())
      return true;

    // if we have reached the end of 'p' but 'm' or 'n' still have some characters left
    if(pIndex == p.length())
      return false;

    boolean b1=false, b2=false;
    if(mIndex < m.length() && m.charAt(mIndex) == p.charAt(pIndex))
      b1 = findSIRecursive(m, n, p, mIndex+1, nIndex, pIndex+1);

    if(nIndex < n.length() && n.charAt(nIndex) == p.charAt(pIndex))
```

You must run your code first

The time complexity of the above algorithm is exponential $O(2^{m+n})$, where 'm' and 'n' are the lengths of the two interleaving strings. The space complexity is $O(m+n)$, the value that is used to store the recursion stack.

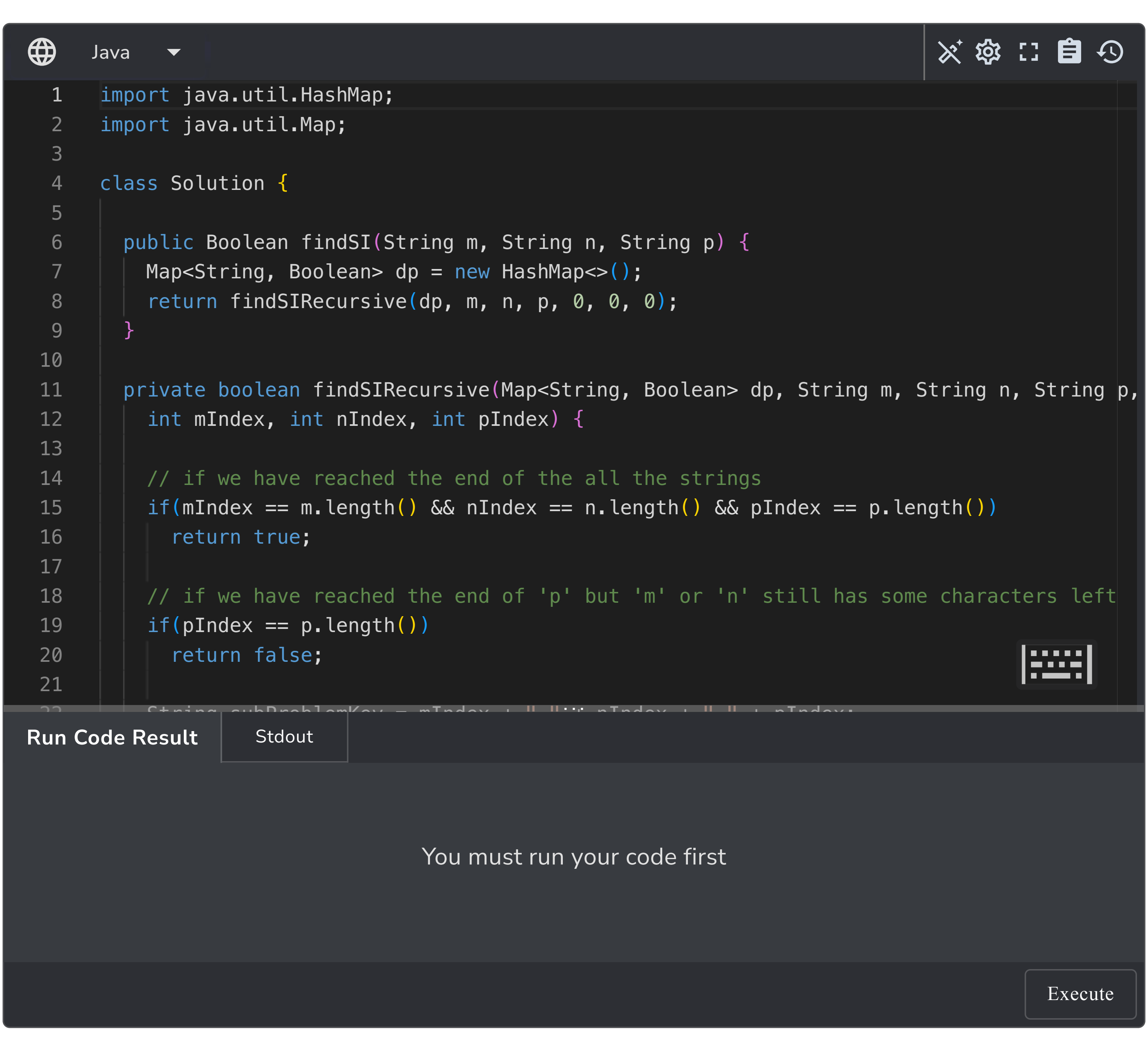
Top-down Dynamic Programming with Memoization

This problem can have overlapping subproblems only when there are some common letters between 'm' and 'n' at the same index. Because whenever we hit such a scenario, we get an option to match with any one of them.

The three changing values in our recursive function are the three indexes `mIndex`, `nIndex`, and `pIndex`. Therefore, we can store the results of all the subproblems in a three-dimensional array. Alternately, we can use a hash-table whose key would be a string (mIndex + "|" + nIndex + "|" + pIndex).

Code

Here is the code:



```java
import java.util.HashMap;
import java.util.Map;

class Solution {

  public Boolean findSI(String m, String n, String p) {
    Map<String, Boolean> dp = new HashMap<>();
    return findSIRecursive(dp, m, n, p, 0, 0, 0);
  }

  private boolean findSIRecursive(Map<String, Boolean> dp, String m, String n, String p,
    int mIndex, int nIndex, int pIndex) {

    // if we have reached the end of the all the strings
    if(mIndex == m.length() && nIndex == n.length() && pIndex == p.length())
      return true;

    // if we have reached the end of 'p' but 'm' or 'n' still has some characters left
    if(pIndex == p.length())
      return false;

```

You must run your code first

Bottom-up Dynamic Programming

Since we want to completely match 'm' and ‘n’ (the two interleaving strings) with 'p', we can use a two-dimensional array to store our results. The lengths of 'm' and 'n' will define the dimensions of the result array.

As mentioned above, we will be tracking separate indexes for 'm', 'n' and 'p', so we will have the following options for every value of `mIndex`, `nIndex`, and `pIndex`:

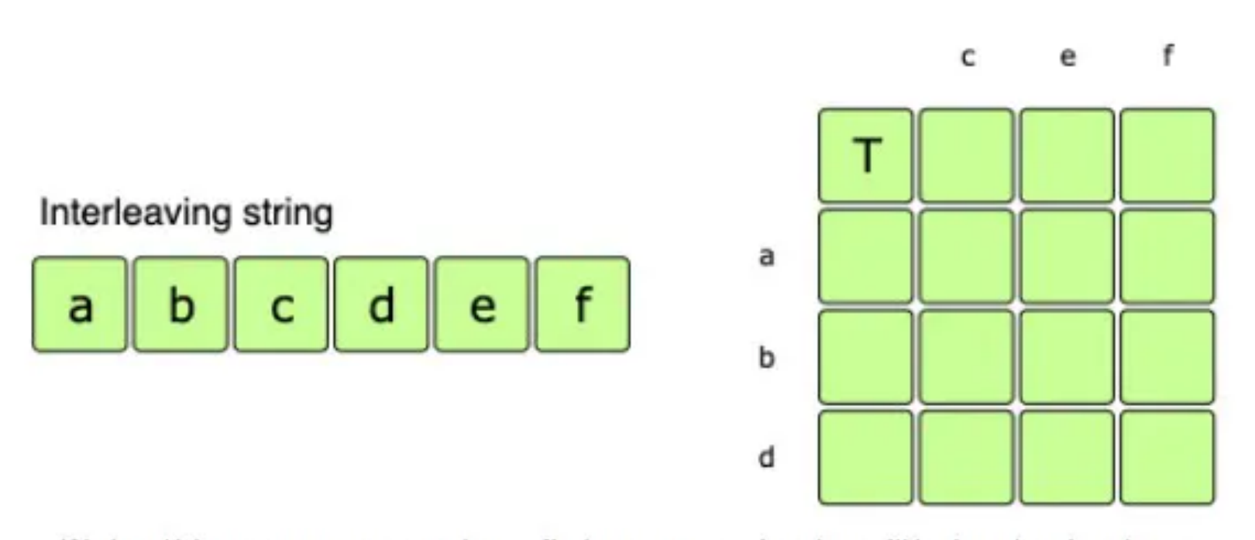
1. If the character `m[mIndex]` matches the character `p[pIndex]`, we will take the matching result up to `mIndex-1` and `nIndex`.
2. If the character `n[nIndex]` matches the character `p[pIndex]`, we will take the matching result up to `mIndex` and `nIndex-1`.

String 'p' will be interleaving strings 'm' and 'n' if any of the above two options is `true`. This is also required as there could be some common letters between 'm' and 'n'.

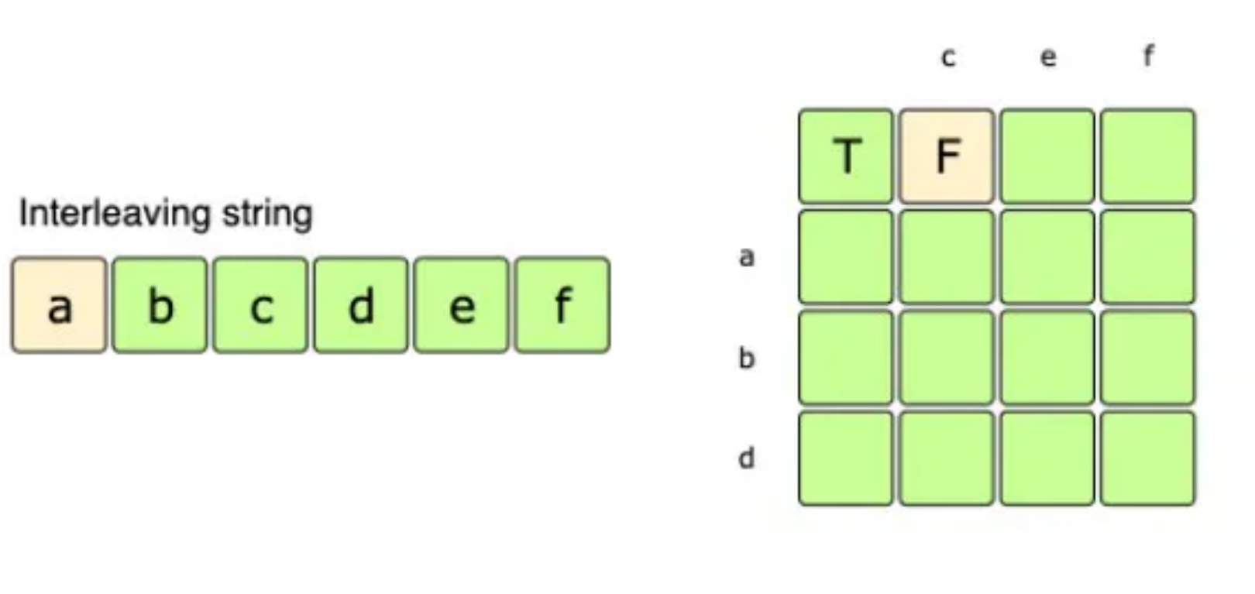
So our recursive formula would look like:

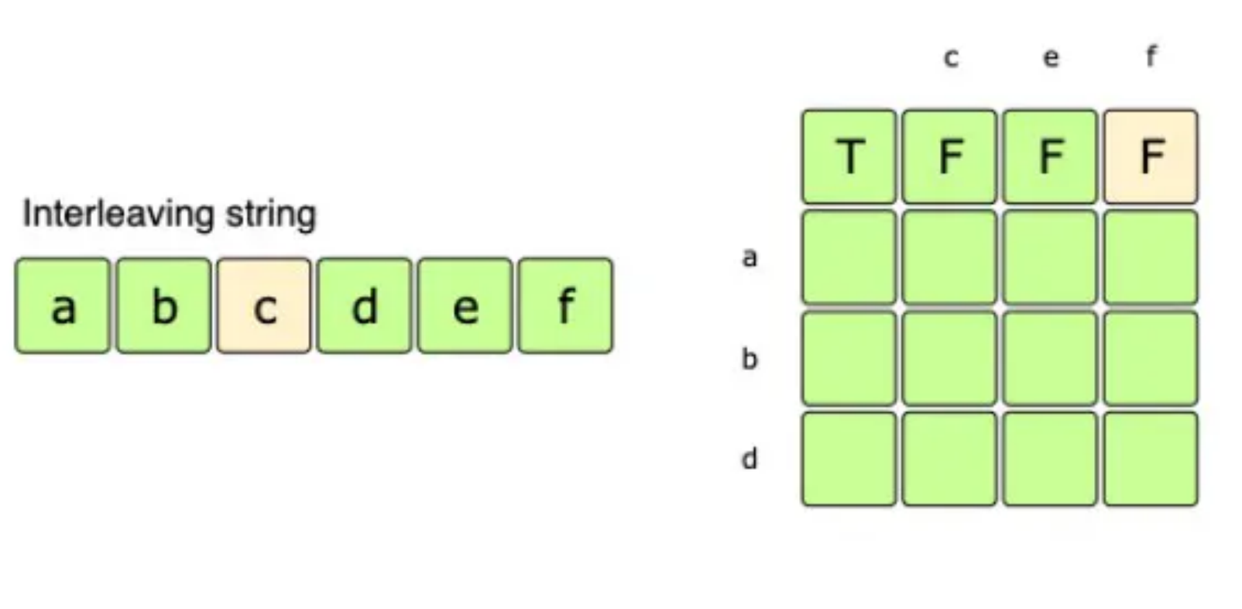
```
dp[mIndex][nIndex] = false
if m[mIndex] == p[pIndex]
  dp[mIndex][nIndex] = dp[mIndex-1][nIndex]
if n[nIndex] == p[pIndex]
 dp[mIndex][nIndex] |= dp[mIndex][nIndex-1]
```

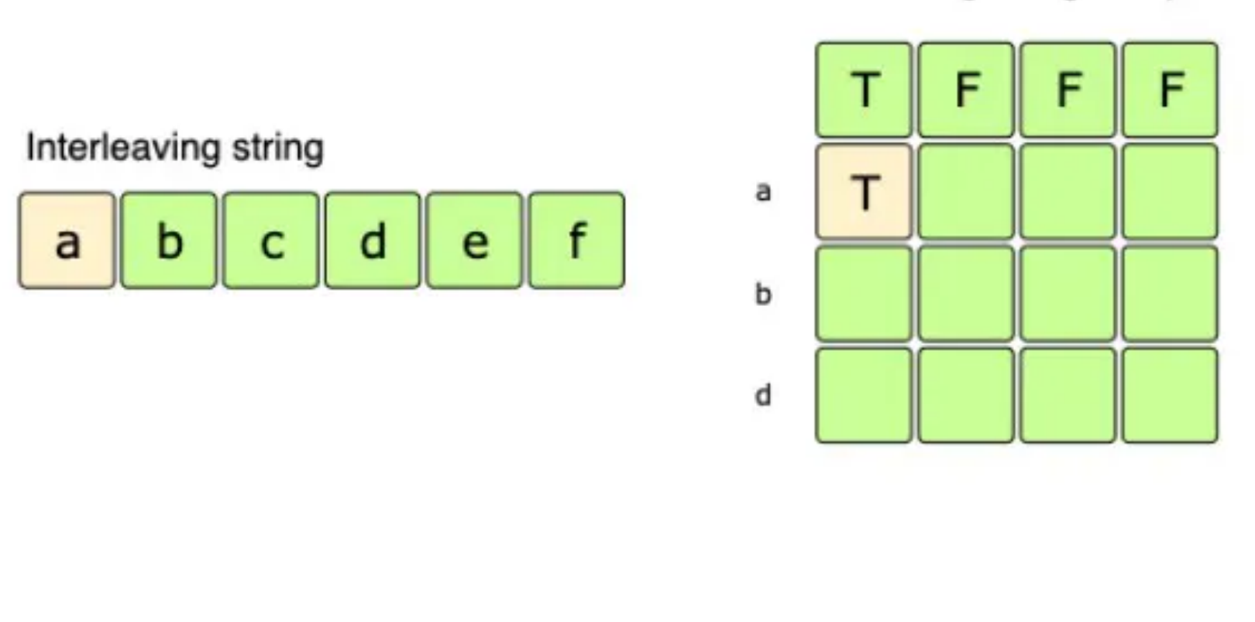
Let's draw this visually:

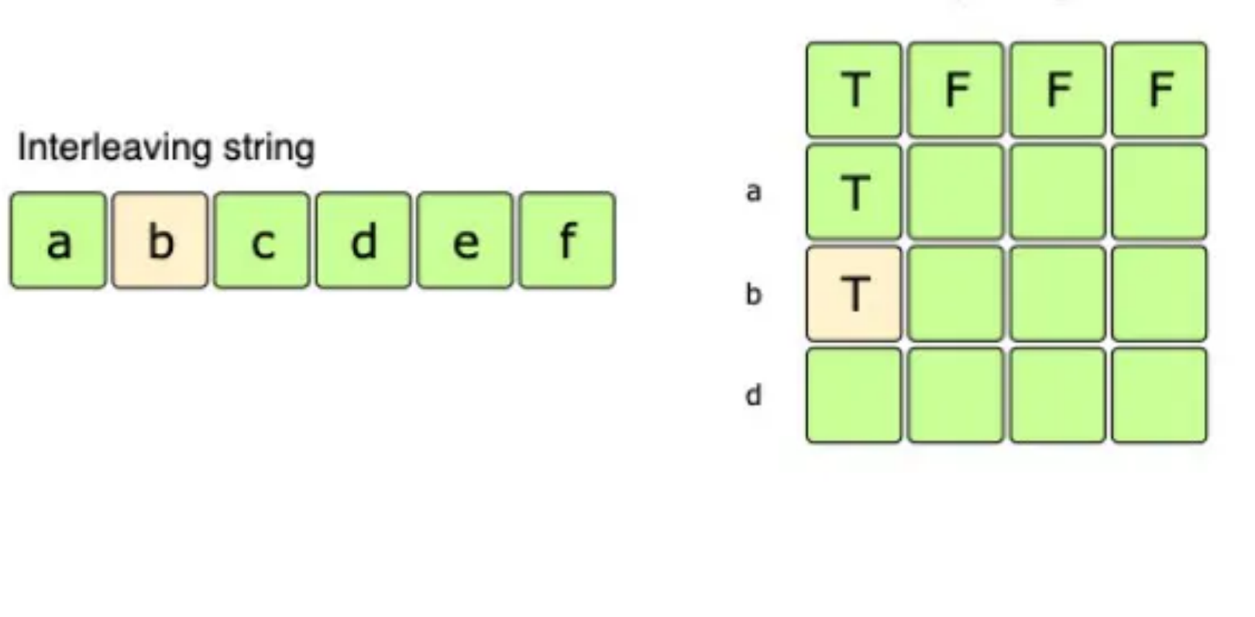


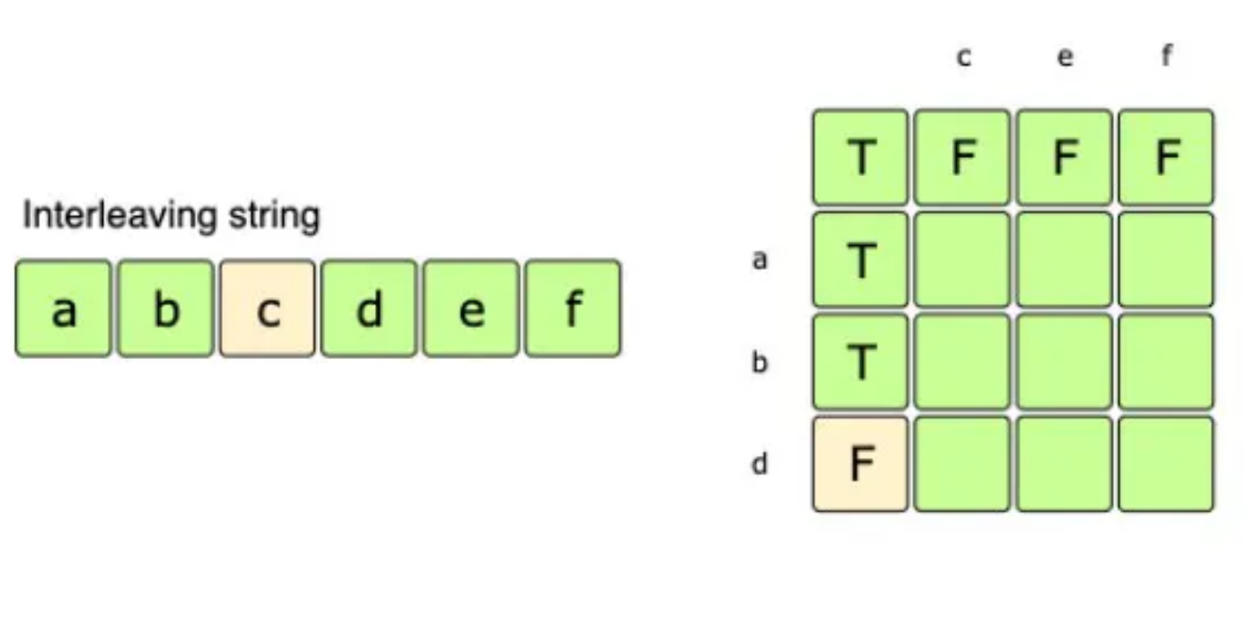
If 'm' and 'n' are empty, we can always find an empty string that will be interleaving them

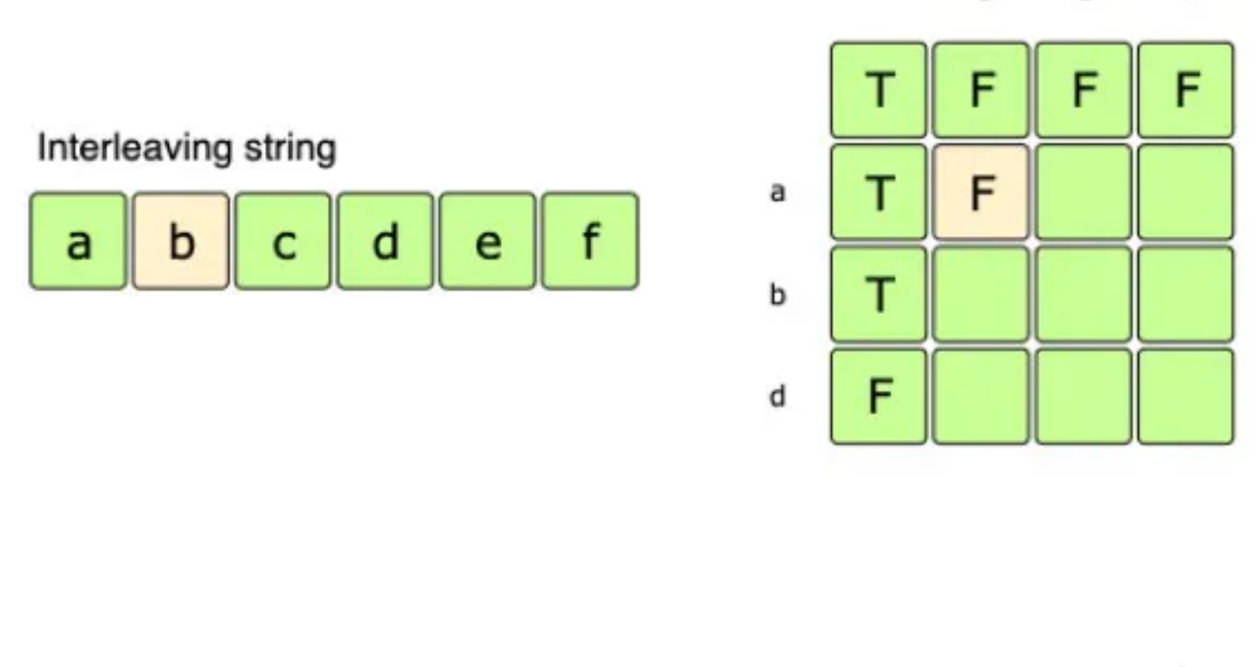


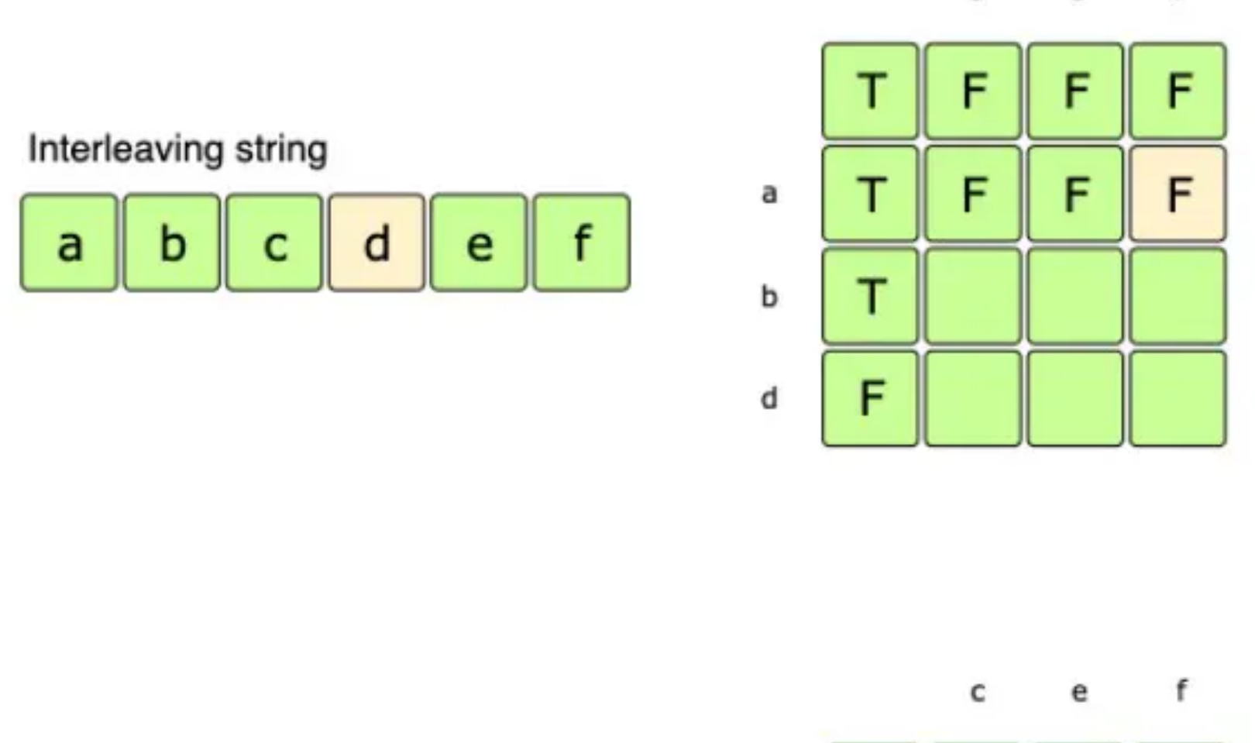


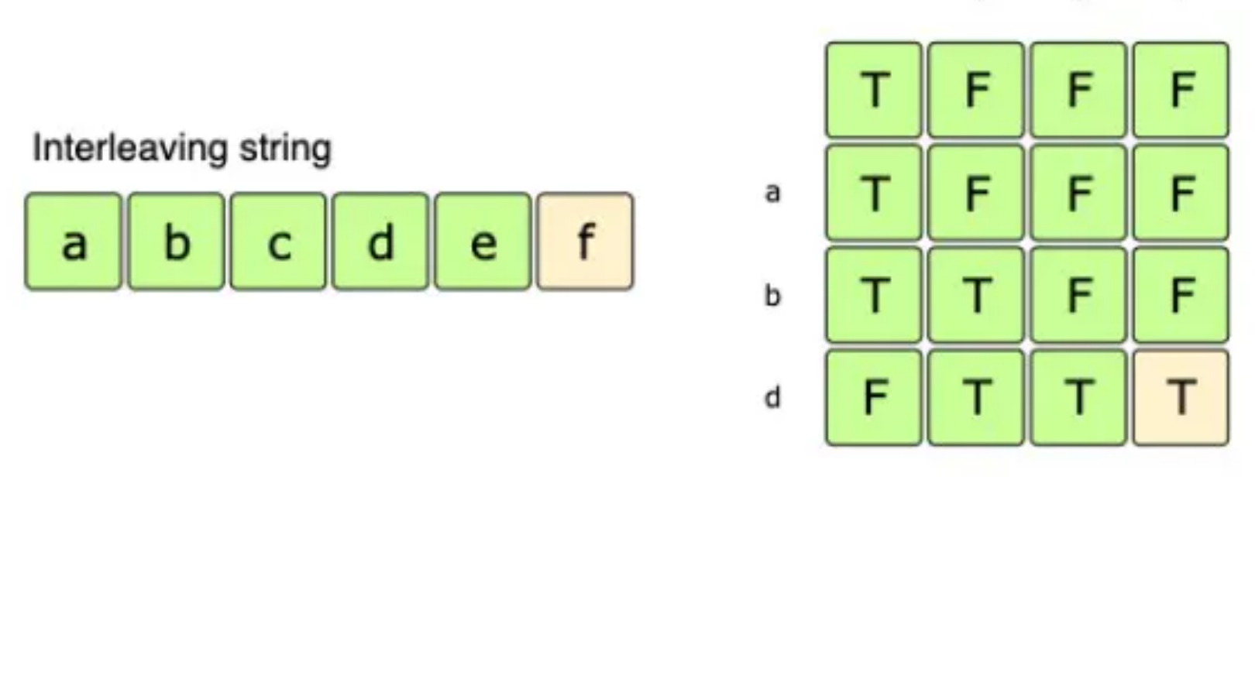






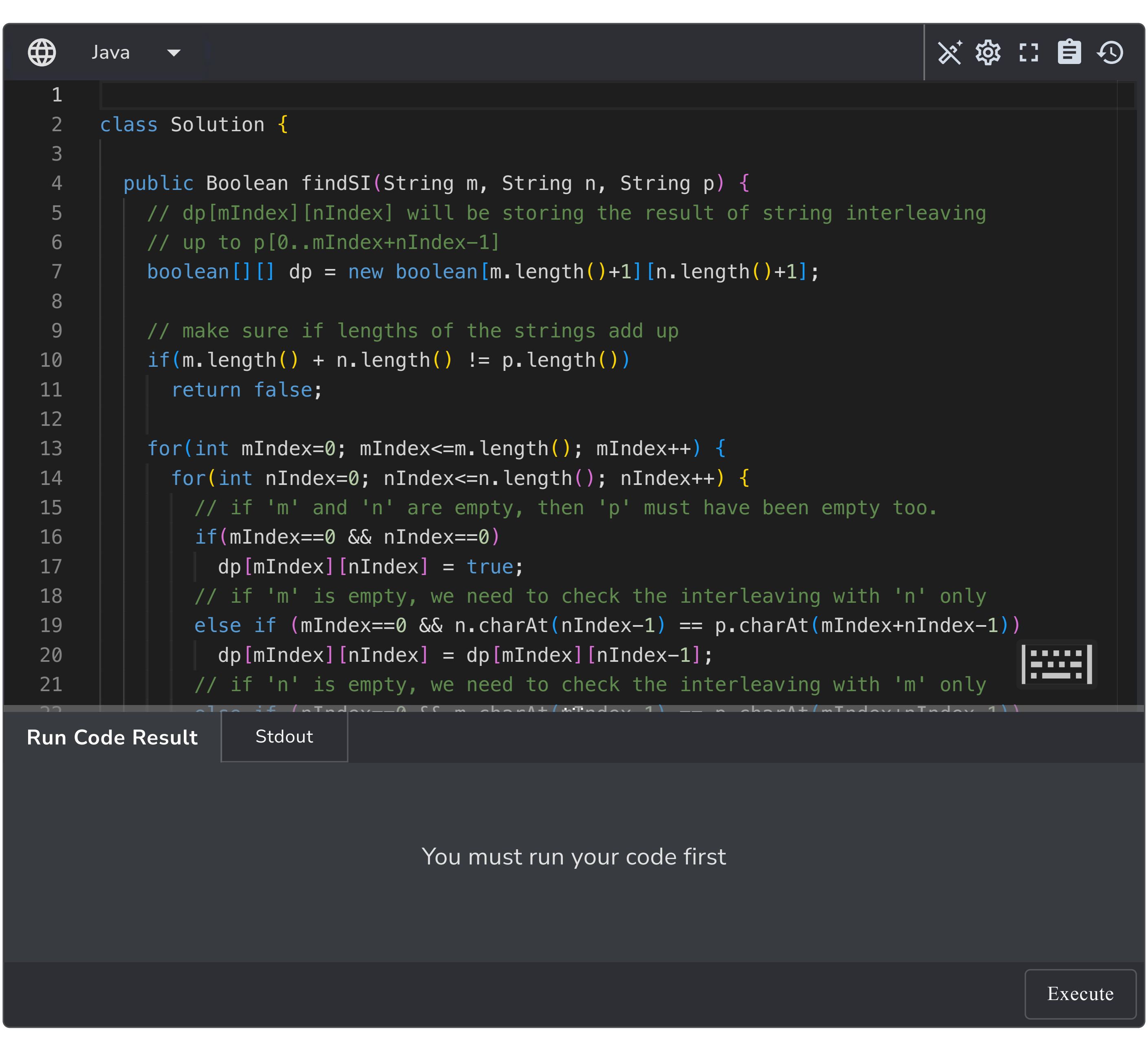






Code

Here is the code for our bottom-up dynamic programming approach:



```java

class Solution {

  public Boolean findSI(String m, String n, String p) {
    // dp[mIndex][nIndex] will be storing the result of string interleaving
    // up to p[0..mIndex+nIndex-1]
    boolean[][] dp = new boolean[m.length()+1][n.length()+1];

    // make sure if lengths of the strings add up
    if(m.length() + n.length() != p.length())
      return false;

    for(int mIndex=0; mIndex<=m.length(); mIndex++) {
      for(int nIndex=0; nIndex<=n.length(); nIndex++) {
        // if 'm' and 'n' are empty, then 'p' must have been empty too.
        if(mIndex==0 && nIndex==0)
          dp[mIndex][nIndex] = true;
        // if 'm' is empty, we need to check the interleaving with 'n' only
        else if (mIndex==0 && n.charAt(nIndex-1) == p.charAt(mIndex+nIndex-1))
          dp[mIndex][nIndex] = dp[mIndex][nIndex-1];
        // if 'n' is empty, we need to check the interleaving with 'm' only
```

You must run your code first

The time and space complexity of the above algorithm is $O(m * n)$, where 'm' and 'n' are the lengths of the two interleaving strings.

← Previous

Next →

Strings Interleaving

Contact Us

☐ Mark as Completed